Draft version October 14, 2026
Typeset using LaTeX **twocolumn** style in AASTeX7.0.1

CLASSIFICATION OF SUPERNOVA LIGHTCURVES FROM THE TRANSIENT NAME SERVER USING MACHINE LEARNING ALGORITHMS

Tyler Donahue[1] and Jack Green[1]

[1] *University of California, San Diego, 92093*

ABSTRACT

With the introduction of the Rubin Observatory, beginning of the Legacy Survey of Space and Time (LSST), and exponentially increasing collection of observational data on transients, the development of autonomous transient classification pipelines will be of particular importance in the coming years. Type Ia SNe in particular comprise a fundamental component of the cosmic distance ladder, and as such, optimization of the classification and analysis of their lightcurves will prove essential to large-data cosmology in the coming decades. Presently, supernovae are classified primarily using spectroscopic data, rather than photometry, because photometric data typically carry higher uncertainty and lack physical markers of supernova classification as utilized in spectroscopic classification. Consequently, the training of models to accurately classify by use of photometry alone, if viable, presents a highly efficient solution to the current large collection of photometry and minimal capacity to take spectroscopic observations by astronomers today. In this study, we utilize the packages numpy, scipy, pandas, matplotlib, scikit-learn, and george to construct classification pipelines for 2327 supernova photometric observations. We do this by using george to model fit light curves, and a random forest machine learning algorithm with cross-validation to test its accuracy. We find that, given our data set, we were able to achieve a maximum accuracy of 73%. Of all SNe light curves, it was able to correctly classify 89% of Type Ib, 80% of type Ic, 73% of type II, and 52% of type Ia.

1. INTRODUCTION

Within the past twenty years, the field of astronomy has undergone significant and fundamental changes, notably so in the scope and capacity of observation and collection of observational data. Highly successful worldwide collaborations like Gaia, TESS, Zwicky Transient Facility (ZTF), Pan-STARRS, and now the Legacy Survey of Space and Time (LSST), among others, all produce observational data on a scale that exceeds human capacity for manual processing and analysis. In transient astronomy, LSST is expected to observe millions of transient events in only the first several years, a significant fraction of which will be supernova photometry. Of all observed supernovae to date, only several thousand have been manually classified by researchers, and most of those are done spectroscopically. In this imminent "Big Data" era of astronomy, autonomous and/or automated pipelines for the classification of transient events will be necessary to keep up with, or at least moderate, the innumerable observations we will make. Numerous astronomers and researchers are developing and supporting such pipelines for spectra, photometry, images, and simulations, using a wide array of machine learning algorithms, statistical methods of analysis, and neural network solutions. However, a constraint remains that the collection of photometric data will far exceed the collection of spectroscopic data; with reliable spectroscopy limited, photometry-only classification will also be necessary.

In this study, we attempt to create a pipeline of our own, as a proof of concept that classification pipelines can be accessible even to students and citizen scientists. We create a machine learning pipeline to classify supernova light curves using only the photometric data of already-classified SNe. The repository that houses our pipeline is available on our GitHub, which can be found in the Appendix below. We first gather this data using the Transient Name Server (TNS) to query ZTF observations from the ALeRCE data broker. Using the gaussian process regression george model fitting package, we then model each supernova light curve in our set, and extract features from those modeled light curves followed by resampling of the feature space using imbalanced-learn's SMOTE resampler to balance the features. We then utilize a random forest algorithm in tandem with cross-validation to classify our dataset.

Email: tydonahue@ucsd.edu, jtg002@ucsd.edu

2. DATA

Our initial dataset consists of a very large table that contains object IDs and other information on more than 190,000 detected transient events, which we downloaded from the TNS database after requesting and being granted an account. Onto this table, we apply a mask to retain only ZTF observations; we do this for consistency, as LSST has not yet released many classified SNe. Observations from ZTF were all originally captured by the Samuel Oschin 40" Telescope at Palomar Observatory, which is the primary telescope used in the ZTF survey. We then apply a second mask, to retain only SNe that are classified as types Ia, Ib, Ic, and II; because the table holds many objects with uncommon or unique classifications, we retain only the four basic SNe types. This was accomplished in the tns_handeling.ipynb notebook located in the Code folder of our repository linked in the appendix.

After applying these masks, we are left with 8105 total supernovae to work with. Of these, 6438 are type Ia, 147 are type Ib, 189 are type Ic, and 1331 are type II. These numbers are sufficient for training the machine learning model. To acquire light curves and other measured values, we query them from the ALeRCE data broker's forced photometry table, using all of our retained object IDs. We then save all of these raw light curves into json files organized by classification, for instantaneous access throughout our model fitting and ML-training processes. All code pertaining to these queries is contained in the alerce_handeling.ipynb notebook located in the Code folder of our repository and all collected data is stored in the Data folder.

3. METHODS

3.1. *Model Fitting with Gaussian Process Regression*

In order to train a machine learning algorithm to classify SNe lightcurves, direct input parameters, as features, are necessary; in order to extract these features from each lightcurve, a standard format of lightcurve model is necessary. That is to say, we need full modeled light curves to extract features, as any raw light curve will vary greatly in both number of data points and error value measurements.

We have chosen to use gaussian process regression through the george model fitting package to fit our light curves because of its relative ease of use, and accessibility to beginners (S. Ambikasaran et al. (2015)). This is accomplished in the fit_george function located in our utils.py file in the Code folder of our repository. We first define the time interval being fitted in the given Modified Julian Date format to be 50 Julian Days before first detection, and 100 Julian Days after final detection. This is to properly capture rise times that may be cutoff from the actual first detection and similarly to capture end behavior for fall time. We then initialize the Matern32 kernel with a metric of 100, because it is optimized for periodic functions. This is helpful because the plateau feature in many type II SN light curves can resemble a periodic function. We initially attempted to use other kernels, such as ExpSquared-Kernel, but visual inspection and comparison of their outputs revealed that Matern32 produces categorically better fits than the available alternatives.

We then utilize the HODLR solver algorithm to solve for the best fit on each light curve. We chose HODLR because it is optimized for linear problems, and is noticeably more efficient in its matrix multiplications than other solvers. We also found that, compared to other solvers, it overall produced better fits for any given light curve. However, the quality of curve fits varies greatly, even with the HODLR solver.

To further our fits accuracy we implemented a rudimentary hyperparameter optimization algorithm as laid out in the george documentation, under the documentation page of the same name (S. Ambikasaran et al. (2015)). We chose to optimize george in this way because it noticeably improved our fits and had a lower run time than implementing a Markov Chain Monte Carlo on the kernel hyperparameters and the time domain hyperparameters.

3.2. *Pipeline Modularization*

Once our code successfully fit some sample light curves, we modularized our entire code base to streamline our pipeline. All TNS-related querying was relegated to its own file, as well as ALeRCE querying to its own. Our utils.py file holds all of the peripherals for calculating statistical values, all of the lightcurve fitting models we attempted to utilize, and our functions for extracting each feature from any given model. These optimizations collectively enabled us to move forward efficiently and allowed us to retroactively apply updates to our data that would propagate up our code stream that improved our efficiency and prevented delays when stuck on data pre-processing prior to implementing the machine learning algorithm.

3.3. *Feature Extraction*

With all of the supernova light curves fitted, we then use these fits to extract features to use as parameters in our machine learning algorithm. Our first parameter is also the simplest to recover: peak brightness. We extract peak brightness by simply taking the value of the highest point in the model fit curve in the

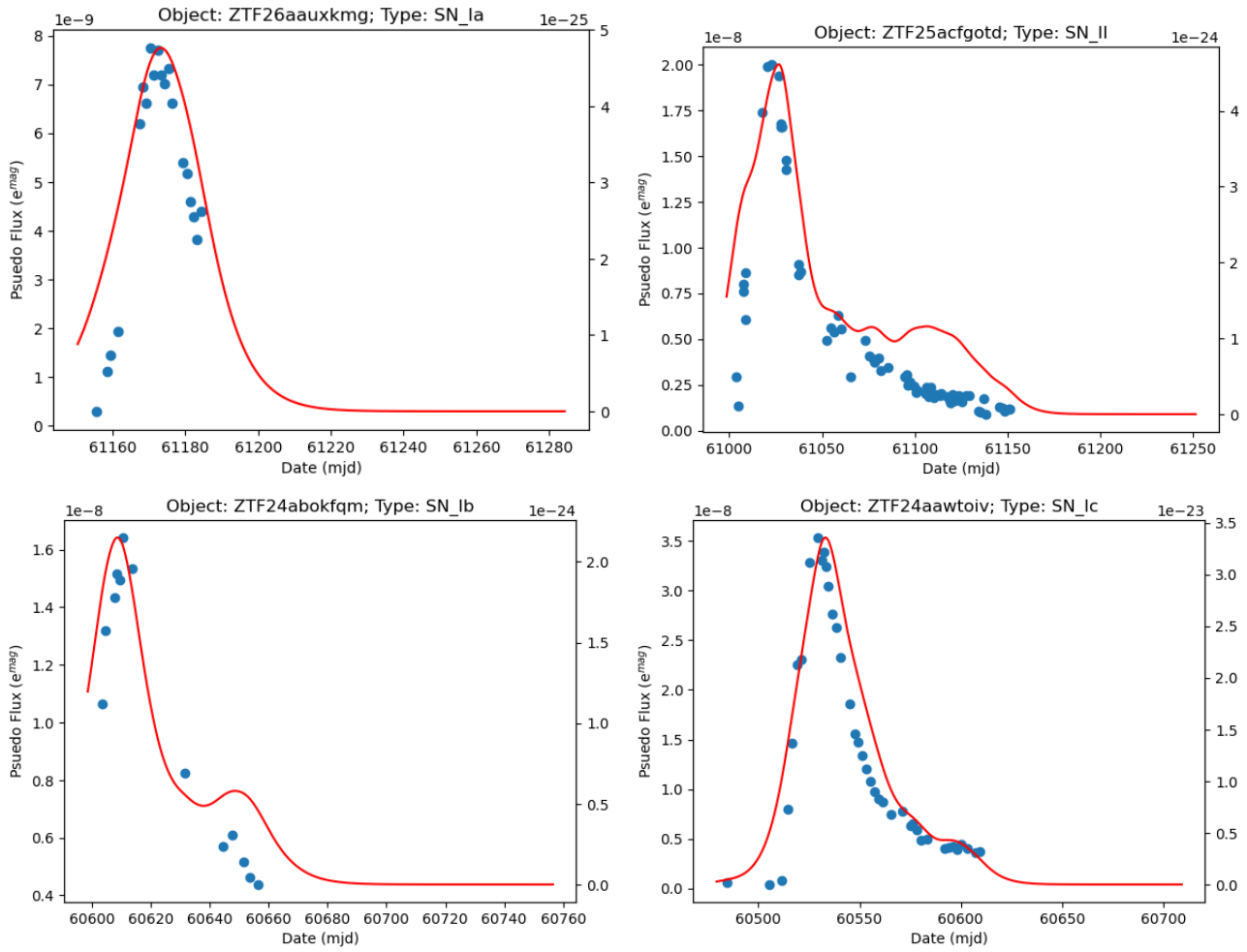


Figure 1. Four raw light curves of the four SNe types we classify, overplotted with the george model fit: type Ia (top left), II (top right), Ib (bottom left) and Ic (bottom right). These four exemplify ideal light curves of each type. Note the relative inaccuracy of the model fit.

light_curve_fitting.ipynb notebook in the Code folder of our repository. We obtain our other two parameters, rise time and fall time, by first using NumPy's gradient function to obtain slope at every point. We then define "rise time" as being the interval between the first index where slope is greater than 1 and the index of peak magnitude, and "fall time" as the inverval between the index of peak magnitude and the index where slope drops lower than -1. We chose these slopes because, in the typical morphology of the fitted curves, a slope of ± 1 visually corresponds closely to where the curve begins rising and finishes falling, which are the inflection points. We implemented these in the rise_time_slope and fall_time_slope functions in our utils.py file in the Code folder of our repository. With these definitions made, we perform feature extraction on every light curve, and save all of these features in a table to one .ecsv file for each SN type located in the Data folder of our repository.

We had initially defined rise and fall time using a more rudimentary set of criteria, wherein the outer bounds of both intervals were set merely to be the first and final index of the model fit, for rise and fall time, respectively. We quickly realized that with these definitions, our extracted rise and fall times were neither physically meaningful nor useful for training the ML classifier in any capacity. These slope-based definitions, though still not ideal, are closely tied to the fit model itself; the padding added to the time axis when generating the fits are even less meaningful, and including them in the rise and fall times actually interferes with the goal of feature extraction.

3.4. *Normalization & Resampling*

Before the machine learning model can be trained, we also normalize our set of fitted curves, and then resample the set in order to increase the number of curves of each type to balance out the overabundance of type Ia SNe

which prevents overfitting. We first encode SNe types from dictionary format into a simple numbered system i.e. SNIa to 0, etc. This is an optimization step for the ML classifier.

Next, we normalize our features using scikit-learn's RobustScaler (F. Pedregosa et al. (2011)). This function is resistant to outlier values because it uses the median absolute deviation to scale the values rather than an outlier value such as in min-max scaling, or scaling from the mean.

To resample our set of curves, and mitigate oversampling on the Ib and Ic light curves, we utilize the imbalanced-learn package's SMOTE resampler tool, set with k=3 neighbors and the default "auto" sampling strategy (G. Lemaître et al. (2017)). This effectively trains a k-nearest clustering algorithm on the labels, and generates synthetic features to fill the parameter space of the given SNe types. This balances the ML-training dataset to have exactly 1750 of each SNe type; this brings our data up to 7000 sets of features and labels total. All of this section is contained within the machine_learning.ipynb notebook located in the Code folder of our repository.

3.5. *ML Methods & Cross-Validation*

For classifying our fitted SNe light curves, we consider and utilize only the best one of three different machine learning algorithms, provided by scikit-learn (F. Pedregosa et al. (2011)). The first that we tried was a support vector machine, from scikit-learn, with all default settings: RBF kernel, which is a squared exponential function, and a degree of three. This resulted in a relatively low accuracy of around 50% to 55%. Rather than adjust hyperparameters, we moved on to the next option. The second ML algorithm we used was a k-nearest-neighbors algorithm. We tried several values of k, between 5 and 100, and ran our feature-label sets of SNe through the algorithm to retrieve similarly poor accuracy for the entire range of k values. Lastly, and most successfully, we utilized scikit-learn's random forest classifier on our dataset, starting from the default random state of 0. For creating our predictions, we run the random forest through scikit-learn's cross-validation. This validates the model by testing it against various selected portions of the training set, rather than through defining dedicated training and testing sets; it also mitigates overfitting. We then calculated the accuracy of these predicted classifications against our resampled labels and plotted them in confusion matrices normalized by row and column to extract completeness and purity (Figure 3). All of this section is contained within the machine_learning.ipynb notebook located in the Code folder of our repository.

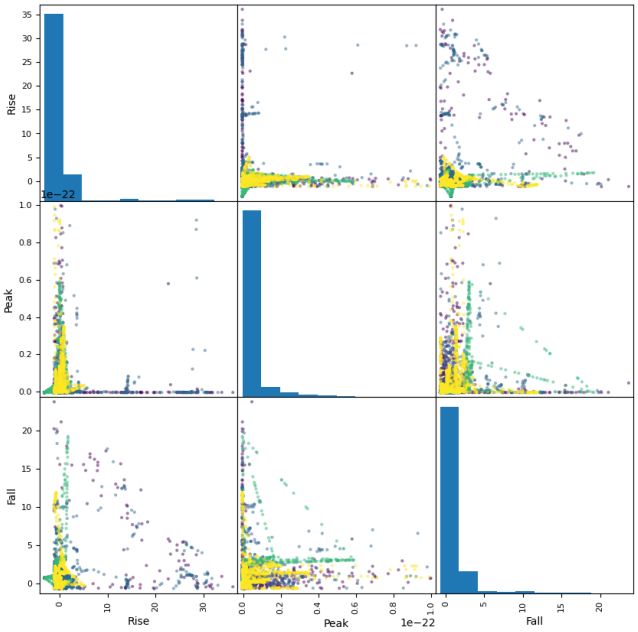


Figure 2. Scatter matrix of our random forest ML classifier and its three features/parameters. Rise time is represented by the top row and left column; peak magnitude by the middle row and column; and fall time by the bottom row and right column. Note the abundance of outlier SNe which distort the resulting scatter matrix. The key takeaway is that manual dataset cleanup is complicated and very time-consuming.

4. DISCUSSION

4.1. *Results*

We report that our most successful ML classifier option, the random forest, resulted in a classification accuracy of 73%. Its highest completeness is 89% when classifying type Ib SNe followed shortly by 80% for SNe Ic and 73% for SNe II. However, our model only reports a completeness of 52% for SNe Ia, and generally confuses them with the other types almost equally, weighted slightly toward type II. We believe this is an artifact of our gaussian process fit because it has trouble distinguishing the plateaus of the other types and the SNe Ia are fit as non-oscillatory gaussian and thus present similar features to the other types when not considering their plateaus. We report purities of 81% for SNe Ib, 77% for SNe Ic, 71% for SNe II, and 63% for SNe Ia. Our model confusing types Ia at a high rate impacts our purity more than our completeness and showcases motivation for choosing a new model in future work.

Our model's reported feature importances and mean decrease in impurity (MDI) are equally split between the rise and fall times of our fit with ∼50% MDI and ∼70% importance. The peak has no weight in the classification, which is odd and is another nail in the coffin of george. We believe this is caused by the lack of redshift which limits us to apparent magnitude; this is not physically linked to the SNe but rather to their distance.

We consider this project to be a relative success. Our classifier, trained on a relatively small dataset and coded and assembled in just short of two months, demonstrates that pursuits like this are entirely reasonable and achievable for students and citizen scientists. Our maximum achieved 73.5% classification accuracy is notable for using photometry only, and we believe this figure demonstrates that spectra-dark classification of SNe is certainly possible.

4.2. *Limitations*

We recognize this study has several key limitations, in both methods and scope. Most significantly, our light curve model fits are overall poor and in need of improvement. Though quick and efficient, the george package is not equipped to handle the high variability of SNe photometry; at the least, not as we have implemented it. Although fits only need to be "good enough" to distinguish from each other, creating better fits overall would most certainly increase our ML classifier's accuracy and allow for more flexibility in classifications.

In addition, our method of feature extraction for training the ML algorithm can be optimized. Due to a lack of access to redshift values, our Peak Magnitude feature is entirely arbitrary; this is because the brightness values provided by ALeRCE are in *apparent* magnitude, rather than absolute. Therefore, we were unable to convert to absolute magnitude, luminosity, or any other absolute brightness scale. Nevertheless, with rise and fall times alone we were still able to achieve nearly 70% accuracy. It is highly likely that, in the case of redshift being an available feature, we could expand our number of *effective* features from two to four, by inclusion of both redshift and calculated peak luminosity. Our choice to use slope m=±1 as outer bounds, though logical through visual inspection alone, is not robust, and is only slightly more physically meaningful than any other choice. As such, a more physical definition of rise and fall time should be implemented if possible.

In addition, the size of our dataset is a major limiting factor. Access to only ∼2300 available light curves with existing SNe type classification inherently limits the model's ability to handle outliers and edge cases. As the number of manual classifications increase with time, and as LSST provides higher quality photometry, it is likely that data access will also enable this project, or projects like it, to become much more effective in the coming years. That, however, constitutes a task for further study.

4.3. *Future Work*

Ultimately, the scope of this project is capable of exceeding current work by an order of magnitude or more. As such, possibilities for future work on this project are numerous. The largest goal is to create better fits on the light curve. Previous studies have defined model functions for light curves that create much better fits (G. Hosseinzadeh et al. (2020)), and use of such functions would certainly increase our ML classifier's accuracy greatly. Following this, more physically and robustly defining features to extract would be prompt and necessary. As it stands, our retrieved feature importances indicate the need of better features; we would be very interested in pursuing this, to acquire more realistic feature importances. These two tasks would likely increase the accuracy of our existing pipeline significantly. Further cleanup of our dataset to remove remaining anomalous outliers, which are most noticeable in our confusion matrices found in Figure 2 above, is necessary. Furthermore, gathering redshift data for our SNe to convert to absolute magnitude will also improve the generated fits.

In addition, we are interested in training a multilayered neural network to classify SNe light curves, and it could manifest in more than one way. We could train a neural network on features, or on the entire light curves. We could also use a neural network to extract features from a set of SNe light curves and then train an ML algorithm on those features instead.

Lastly, the prospect of creating an autonomous pipeline, based on our pipeline here, could prove very interesting. We would be interested in creating a pipeline to query from data brokers and classify newly-observed SNe light curves without human micromanagement, in real time. Such tools, once their classifier accuracy is pinpoint, would prove invaluable to astronomers and would relieve the burden of manual classification.

5. APPENDIX

Our GitHub repository: https://github.com/tyler-donahue/Astro-154_Final_Project

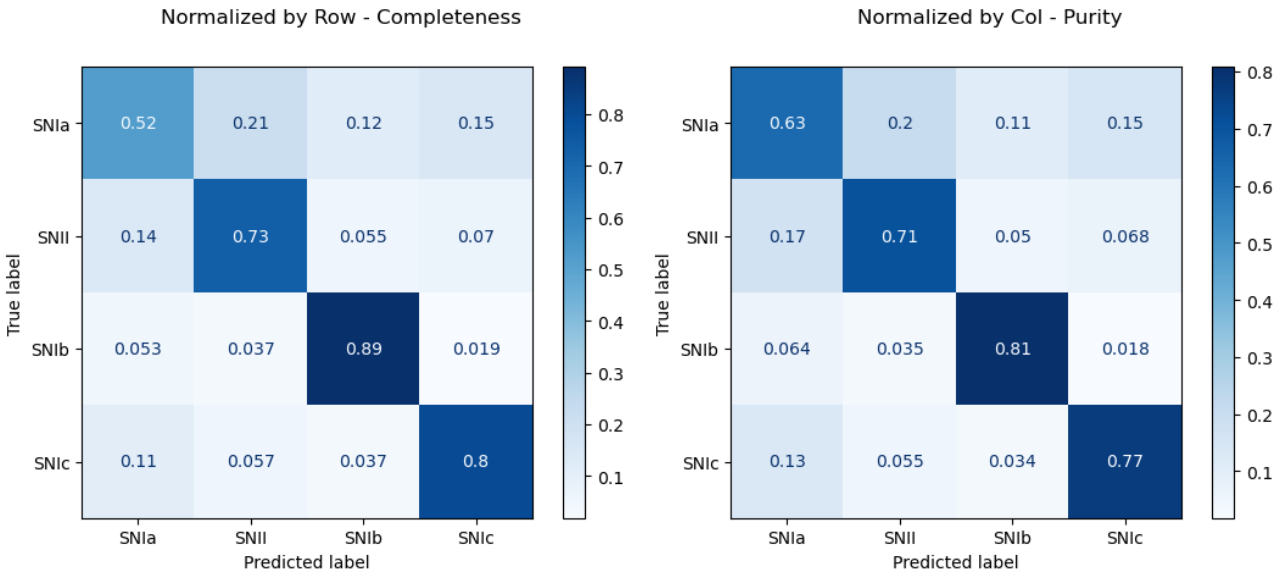


Figure 3. Confusion matrices of our pipeline's classifications, against classification from literature, normalized by row (left) and by column (right). Note the high success in classifying Ib and Ic SNe, and the confusion of Ia for other types. We believe this is caused by Ia light curves having weaker extractable features compared to the other types, being essentially only a rise and fall.

REFERENCES

Ambikasaran, S., Foreman-Mackey, D., Greengard, L., Hogg, D. W., & O'Neil, M. 2015, IEEE Transactions on Pattern Analysis and Machine Intelligence, 38, 252, doi: 10.1109/TPAMI.2015.2448083

Hosseinzadeh, G., Dauphin, F., Villar, V. A., et al. 2020, The Astrophysical Journal, 905, 93, doi: 10.3847/1538-4357/abc42b

Lemaître, G., Nogueira, F., & Aridas, C. K. 2017, Journal of Machine Learning Research, 18, 1. http://jmlr.org/papers/v18/16-365.html

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. 2011, Journal of Machine Learning Research, 12, 2825. http://jmlr.org/papers/v12/pedregosa11a.html